

บทที่ 4

การออกแบบและการพัฒนาโปรแกรม

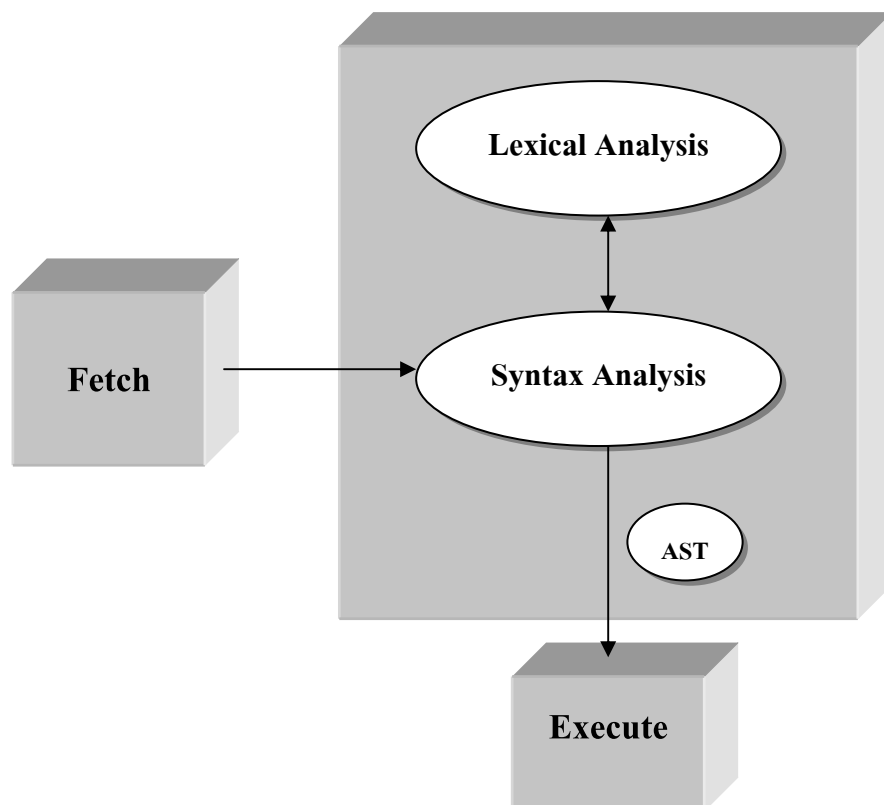
ในการออกแบบและพัฒนาอินเตอร์พรีเตอร์ไพธอนขนาดเล็กนั้น เราจะสามารถมองโครงสร้างและส่วนประกอบต่างๆ ของโปรแกรมได้อยู่ทั้งหมด 3 ส่วนใหญ่นั้นก็คือ

- ส่วนของการอ่านคำสั่งเข้ามาทีละ 1 ชุดคำสั่ง (Fetch)
- ส่วนของการวิเคราะห์คำสั่ง (Analyze)
- ส่วนของการกระทำตามคำสั่ง (Execution)

โดยที่ส่วนที่มีความยืดหยุ่นและซับซ้อนมากที่สุดจะเป็นในส่วนของการวิเคราะห์คำสั่ง เนื่องจากว่าในการวิเคราะห์คำสั่งนั้นจะมีองค์ประกอบอีกถึงสามส่วนคือ

- ส่วนของการตรวจจับคำ (Lexical Analysis)
- ส่วนตรวจสอบประโยค (Syntax Analysis)
- ส่วนพาร์ซทรี (Parse Tree)

ซึ่งเมื่อเรามองภาพขององค์ประกอบทั้งหมดของโปรแกรมแล้วก็จะได้ออกมาดังรูปนี้



รูปที่ 4-1 แสดงองค์ประกอบของอินเตอร์พรีเตอร์ไพธอนขนาดเล็ก

4.1 การออกแบบและสร้างส่วนของการอ่านคำสั่ง

ในการอ่านคำสั่งเข้ามาทีละคำสั่งนั้น อันที่จริงแล้วจะต้องสามารถทราบได้ก่อนว่าคำสั่ง และไวยากรณ์ต่างๆ ของภาษาไพธอนส่วนที่จะนำมาใช้นั้นมีอะไรบ้าง โดยการสร้างให้อยู่ในรูปของ เรกกูลาร์เอ็กซ์เพรสชัน และบีเอ็นเอฟขึ้นมาทั้งหมดเท่าที่ต้องการใช้ จากนั้นก็จะทำการสร้างส่วนที่รับข้อมูลทั้งหมดเข้ามายังส่วนของการอ่านคำสั่ง แล้วอ่านอักขระออกมาทีละตัว เข้าไปเรื่อยๆ จนส่วนของการตรวจสอบประโยคตรวจพบว่าสิ้นสุดคำสั่งแล้วก็จะนำไปกระทำการสร้างทรีต่อไป

สรุปว่าอันที่จริงแล้วการสร้างส่วนของการอ่านคำสั่งนั้นไม่มีส่วนสำคัญอะไรมากเท่าไรเพียงแค่อ่านอักขระเข้ามาทีละตัวเรื่อยๆ จนหมดข้อมูลเท่านั้นเอง

4.2 การออกแบบและสร้างส่วนของการตรวจจับคำ และการตรวจสอบประโยค

ส่วนของการตรวจจับคำ และส่วนของการตรวจสอบประโยคนั้น อันที่จริงแล้วจะไม่สามารถแยกกันทำงานได้เลย เนื่องจากว่าจะทราบว่าประโยคถูกต้องหรือไม่ ก็จำเป็นต้องทราบว่าแต่ละคำในประโยคนั้นถูกต้องหรือไม่เสียก่อน อาจกล่าวได้ว่าส่วนของการตรวจสอบประโยคนั้นจะไปถามส่วนการตรวจจับคำอีกทีว่าคำที่มีอยู่ในประโยคนั้นถูกต้องหรือไม่ และเมื่อถูกต้องทั้งหมดแล้วส่วนการตรวจสอบประโยคจึงจะทำการตรวจสอบว่าประโยคนั้นถูกต้องหรือไม่โดยการออกแบบนั้นจะทำความคุ้นกันไปทั้งสองส่วน

4.2.1 การออกแบบ เรกกูลาร์เอ็กซ์เพรสชัน

การออกแบบเรกกูลาร์เอ็กซ์เพรสชันนั้น เราจะทำการแยกออกจากกันได้เป็นสองส่วนใหญ่ๆ คือ

- คำที่มีลักษณะคงที่
- คำที่มีลักษณะที่สามารถเปลี่ยนแปลงได้

4.2.1.1 คำที่มีลักษณะคงที่

คำที่มีลักษณะคงที่นั้น ก็จะหมายถึงกลุ่มของตัวอักษรในภาษาไพธอนที่เป็นคำสงวนต่างๆ รวมถึงเครื่องหมายการกระทำต่างๆ ซึ่งเราสามารถที่จะแทนเรกกูลาร์เอ็กซ์เพรสชันด้วยอักขระของคำนั้นๆ ได้เลย เช่น

คำว่า “if” เราสามารถเขียนเรกกูลาร์เอ็กซ์เพรสชันได้ดังนี้

IF “if”

ซึ่งหมายความว่า เรกกูลาร์เอ็กซ์เพรสชันชื่อว่า IF ใช้แทนอักษร i และ f ติดกันเท่านั้น หรืออย่างเช่น

เครื่องหมายบวก “+” เราสามารถเขียนเรกกูลาร์เอ็กซ์เพรสชันได้ดังนี้

ADD “+”

ซึ่งหมายความว่า เรกกูลาร์เอ็กซ์เพรสชันชื่อว่า ADD ใช้แทนอักษร “+” นั้นเอง

ซึ่งเมื่อลองทำการสร้างเรกูลาร์เอ็กซ์เพรสชันแบบที่มีลักษณะคงที่ในภาษาไพธอนเท่าที่จำเป็นต้องใช้ในการทำอินเตอร์พรีเตอร์ขนาดเล็กนี้ก็จะได้ออกมาทั้งหมดเป็นดังนี้

ชื่อ	เรกูลาร์เอ็กซ์เพรสชัน
EQU	"=="
GTE	">="
LTE	"<="
NE1	"<"
NE2	"!="
<	"<"
>	">"
OR	"or"
AND	"and"
ADDE	"+="
SUBE	"-="
MULE	"*="
DIVE	"/=
POWE	"**="
MODE	"0/"
POW	"**"
=	"="
+	"+"
-	"_"
~	"~"
*	"*"
/	"/"
%	"0/"
;	","
:	"."
(	"("
)	")"
[	"["
]	"]"
,	","

IS	“is”
IN	“in”
NOT	“not”
IF	“if”
ELSE	“else”
ELIF	“elif”
WHILE	“while”
FOR	“for”
BREAK	“break”
CONTINUE	“continue”
PASS	“pass”
RETURN	“return”
PRINT	“print”
DEL	“del”
RANGE	“range”
LEN	“len”
URL	“openURL”

ตารางที่ 4-1 ตารางเรกูลาร์เอ็กซ์เพรสชันที่มีลักษณะคงที่ที่ได้ทำการออกแบบ

4.2.1.1 คำที่มีลักษณะสามารถเปลี่ยนแปลงได้

คำที่มีลักษณะที่สามารถเปลี่ยนแปลงได้นั้นก็จะหมายถึงกลุ่มของตัวอักษรที่เป็นค่า (value) ซึ่งไม่จำเป็นว่าจะต้องมีค่าเท่านั้นเท่านี้ตายตัวแต่ค่าต่างๆ ทุกค่ามันจะมีรูปแบบ (format) ของค่าชนิดต่างๆ อยู่ซึ่งแตกต่างกันออกไป ฉะนั้นเราจะต้องศึกษาหาความสัมพันธ์ภายในคำนั้นๆ ว่าจะมีลักษณะเป็นอย่างไร ตัวอย่างเช่น

ตัวเลขตัวหนึ่งตัวใดๆ ก็จะสามารถแทนด้วยเรกูลาร์เอ็กซ์เพรสชันได้ดังนี้

digit [0-9]

ซึ่งมีความหมายว่าเรกูลาร์เอ็กซ์เพรสชันชื่อว่า digit ใช้แทนตัวเลข 0, 1, 2, 3, 4, 5, 6, 7, 8 หรือ 9 ตัวใดตัวหนึ่งก็ได้

หรืออย่างเช่น เลขฐานแปด ก็จะสามารถแทนด้วยเรกูลาร์เอ็กซ์เพรสชันได้ดังนี้

octdigit [0-7]

octinteger 0{octdigit}+

โดยการที่เราจะเขียนเรกกูลาร์เอ็กซ์เพรสชันเพื่อแทนความหมายของอะไรก็ตามที่เริ่มจะซับซ้อนขึ้นเราก็จำเป็นที่จะต้องสร้างเรกกูลาร์เอ็กซ์เพรสชันขึ้นมามากกว่าหนึ่งตัวมาสื่อความหมายร่วมกัน โดยจากตัวอย่างข้างต้นจะประกอบไปด้วยเรกกูลาร์เอ็กซ์เพรสชันทั้งหมดสองตัวคือ

octdigit [0-7]

ซึ่งหมายถึงตัวเลข 0, 1, 2, 3, 4, 5, 6 หรือ 7

และ

octinteger 0{octdigit}+

ซึ่งหมายถึงตัวเลขที่ขึ้นต้นด้วยเลขศูนย์ "0" แล้วตามด้วยเลขอะไรก็ได้ตั้งแต่ 0 ถึง 7 อย่างน้อยหนึ่งตัว

ซึ่งเมื่อลองทำการสร้างเรกกูลาร์เอ็กซ์เพรสชันแบบที่มีลักษณะสามารถเปลี่ยนแปลงได้ในภาษาไพธอนเท่าที่จำเป็นต้องใช้ในการทำอินเตอร์พรีเตอร์ขนาดเล็กนี้ก็จะได้ออกมาทั้งหมดเป็นดังนี้

ชื่อ	เรกกูลาร์เอ็กซ์เพรสชัน
identifier	(\{letter\} _)(\{letter\} \{digit\} _)*
letter	(\{lowercase\} \{uppercase\})
lowercase	[a-z]
uppercase	[A-Z]
digit	[0-9]
integer	(\{decimalinteger\} \{octinteger\} \{hexinteger\})
longinteger	\{integer\}[lL]
intpart	\{digit\}+
fraction	"\.\{digit\}+
exponent	[eE][+-]? \{digit\}+
string	'\{stringitem1\}*\{stringitem2\}*\'
stringitem1	(\{stringchar1\} \{escapeseq\})
stringitem2	(\{stringchar2\} \{escapeseq\})
stringchar1	[^\n]
stringchar2	[^\n]
escapeseq	"\"[40-176]
decimalinteger	\{nonzerodigit\} \{digit\}* 0
octinteger	0{octdigit}+

hexinteger	0[xX]{hexdigit}+
nonzerodigit	[1-9]
octdigit	[0-7]
hexdigit	({digit} [a-f] [A-F])

ตารางที่ 4-2 ตารางเรกูลาร์เอ็กซ์เพรสชันที่สามารถเปลี่ยนแปลงได้ที่ได้ทำการออกแบบ

โดยเมื่อเราได้ทำการกำหนดเรกูลาร์เอ็กซ์เพรสชันทั้งหมดของคำที่เราต้องการใช้ในอินเทอร์พรีเตอร์ขนาดเล็กแล้วเราก็จะทำการออกแบบส่วนที่เกี่ยวข้องกันซึ่งก็คือส่วนของตรวจสอบประโยค

4.2.2 การออกแบบ บีเอ็นเอฟ

บีเอ็นเอฟนั้นคือการสร้างประโยคของคำสั่งหรือการกระทำต่างๆ ที่จำเป็นต้องใช้ในอินเทอร์พรีเตอร์ โดยจะต้องเริ่มสร้างจากส่วนของประโยคก่อน ซึ่งไม่มีส่วนเกี่ยวข้องกับประโยคอื่น หรือเรียกได้ว่าเป็นชิ้นส่วนหนึ่งของประโยคที่ย่อยที่สุด และไม่มีส่วนอื่นที่ซ่อนเข้าไปอีกแล้ว เช่น

literal → INTEGER | STRING | LONGINTEGER

ซึ่งจากตัวอย่างข้างต้นจะหมายถึง บีเอ็นเอฟ ที่ชื่อว่า literal นั้นเป็นส่วนหนึ่งของประโยคที่อาจจะนำไปใช้กับประโยคใดๆ ก็ได้ที่มีส่วนประกอบของประโยคเป็นเลข integer, สตริงค์ หรือ long integer

เมื่อเราทำการกำหนดส่วนย่อยของประโยคได้ทั้งหมดแล้ว เราก็จะนำส่วนประกอบต่างๆ เหล่านี้มารวมกันเป็นประโยคที่ใหญ่ขึ้น ซึ่งเมื่อรวมแล้วก็ตามทีก็ยังสามารถนำประโยคที่ได้จากการรวมของส่วนย่อย ไปสร้างเป็นประโยคที่ใหญ่ขึ้นและมีความหมายที่ซับซ้อนขึ้นต่อไป เช่น

augmented_stmt → target augop expression_list

จากตัวอย่างจะหมายถึง บีเอ็นเอฟ ที่ชื่อว่า augmented_stmt นั้นประกอบไปด้วย บีเอ็นเอฟ อื่นๆ ที่เป็นส่วนย่อยของประโยคที่ชื่อว่า target, augop และ expression_list ซึ่งในส่วนย่อยๆ ของ augmented_stmt เองก็อาจจะประกอบไปด้วยส่วนย่อยเข้าไปอีกก็เป็นได้เช่น บีเอ็นเอฟ ที่ชื่อว่า target ก็จะมีลักษณะเป็นดังต่อไปนี้

target → IDENTIFIER | subscription

จะเห็นได้ว่า target นั้นจะประกอบไปด้วย identifier หรืออาจจะเป็นส่วนย่อยของประโยคที่ชื่อว่า subscription ก็ได้ ซึ่ง subscription ก็จะประกอบไปด้วยประโยคย่อยอื่นๆ ไปอย่างนี้เรื่อยๆ จนกว่าจะถึงส่วนย่อยที่สุดที่ไม่ได้ประกอบไปด้วยส่วนของประโยคใดๆ อีกแล้ว

ซึ่งเมื่อลองทำการสร้างบีเอ็นเอฟในภาษาไพธอนเท่าที่จำเป็นต้องใช้ในการทำอินเตอร์พรีเตอร์ขนาดเล็กขึ้นมาแล้วก็จะพบว่าจะได้ บีเอ็นเอฟ ออกมาเป็นจำนวนมาก จึงจะได้ขอยกตัวอย่างมาเพียงบางตัวดังนี้

ชื่อ	บีเอ็นเอฟ
Primary	atom target
atom	literal enclosure
target	IDENTIFIER subscription
literal	INTEGER STRING LONGINTEGER
subscription	primary '[' expression_list ']
enclosure	'(' expression_list ')' '[' expression_list ']
compound_stmt	if_stmt while_stmt for_stmt
if_stmt	IF expression ':' suite elif_stmt else_stmt
elif_stmt	ELIF expression ':' suite elif_stmt
else_stmt	ELSE ':' suite <empty string>
while_stmt	WHILE expression ':' suite else_stmt
for_stmt	FOR target IN expression_list ':' suite else_stmt
range_stmt	RANGE '(' expression ')'
len_stmt	LEN '(' expression_list ')'
expression_stmt	expression_list
expression_list	expression expression_
expression_	',' expression expression_ ';' <empty string>
expression	or_test
or_test	and_test or_test OR and_test
and_test	not_test and_test AND not_test
not_test	comparison NOT not_test

ตารางที่ 4-3 ตารางบีเอ็นเอฟบางส่วนของภาษาไพธอนที่ได้ทำการออกแบบ

ทั้งเรกูลาร์เอ็กซ์เพรสชัน และบีเอ็นเอฟ ที่เราได้ทำการสร้างขึ้นทั้งหมดนั้นจะมีส่วนเกี่ยวข้องกันอย่างมาก เนื่องจากหาก เรกูลาร์เอ็กซ์เพรสชันตัวใดมีการเปลี่ยนแปลงเกิดขึ้น ย่อมส่งผลทำให้ส่วนของประโยคที่มีการใช้ เรกูลาร์เอ็กซ์เพรสชันตัวนั้น เปลี่ยนตามไปด้วย ฉะนั้นในบางครั้งในช่วงการ

ออกแบบอยู่นั้น เมื่อได้ข้อสรุปออกมา แล้วนำไปทำการเขียนโปรแกรม ก็อาจพบผลที่ไม่เป็นไปตามที่ต้องการซึ่งก็จะต้องกลับมาแก้ไขทั้งเรกกูลาร์เอ็กซ์เพรสชันและ บีเอ็นเอฟด้วย จึงทำให้โครงสร้างของภาษาเป็นสิ่งที่สำคัญมาก คือต้องทำการสร้างให้ดี เพราะการแก้ไขแต่ละครั้งนั้นจะส่งผลกระทบต่อส่วนต่างๆ เป็นอย่างมาก

4.2.3 การออกแบบเรกกูลาร์เอ็กซ์เพรสชันและบีเอ็นเอฟ สำหรับทำงานร่วมกับบราวเซอร์ที่โปรแกรมได้

เนื่องจากการพัฒนาและจัดสร้างอินเตอร์พรีเตอร์ในการวิจัยครั้งนี้มีจุดประสงค์ที่จะทำมาเพื่อนำไปใช้กับบราวเซอร์ที่พัฒนาขึ้นให้สามารถโปรแกรมได้ ฉะนั้นด้วยตัวภาษาไพธอนเองนั้นจะไม่มีชุดคำสั่งที่จะสามารถสั่งบราวเซอร์ให้ทำงานได้ดังนั้นจึงต้องมีการเพิ่มเติมคำสั่งการทำงานเพื่อให้เป็นประโยชน์ในการทำงานร่วมกับบราวเซอร์

แต่ด้วยความจำกัดของภาษาจาวาบนโทรศัพท์มือถือ (J2ME : java 2 micro edition) ที่ใช้สร้างตัวบราวเซอร์ขึ้นมานั้น ยังมีความสามารถไม่มากพอที่จะทำงานหลายๆ อย่างได้อย่างเต็มที่ เช่น ส่ง short message, ส่งอีเมลล์ (e-mail) หรือติดต่อกับพอร์ตอินฟราเรด เป็นต้น ซึ่งเป็นผลทำให้ไม่สามารถสร้างคำสั่งที่เกี่ยวข้องกับจุดต่างๆ ดังที่กล่าวมาแล้วได้ ทำให้ในการพัฒนาครั้งนี้ได้ออกแบบคำสั่งที่ใช้งานร่วมกับบราวเซอร์ไว้เพียงคำสั่งเดียวก็คือคำสั่งในการสั่งให้บราวเซอร์เปิดเว็บไซต์ (web site) หรือเว็บเพจ (web page)

โดยที่คำสั่งที่ใช้สำหรับเปิดเว็บไซต์นั้นจะสามารถที่จะกำหนดเวลาในการเปิดได้ ฉะนั้นก็จะมี การเพิ่มเรกกูลาร์เอ็กซ์เพรสชัน ที่ใช้อ้างถึงชนิดข้อมูลชนิดใหม่ เรียกว่าไทม์ (time) ขึ้น

4.2.3.1 ชนิดข้อมูลชนิดไทม์

ข้อมูลชนิดไทม์นั้นจะแบ่งออกได้เป็นสองลักษณะคือ

- Time Point หมายถึงเป็นการระบุเวลาขณะใดขณะหนึ่ง เช่น [17:08:34|15/03/2003]
- Time Interval หมายถึงการระบุจำนวนของเวลา เช่น 2day ซึ่งหมายถึงเป็นจำนวนเวลา 2 วัน

จากลักษณะของข้อมูลชนิดไทม์ดังที่ได้กล่าวมาแล้ว จึงสามารถออกแบบเรกกูลาร์เอ็กซ์เพรสชันสำหรับลักษณะของข้อมูลชนิดไทม์ได้เป็นดังนี้

ชื่อ	เรกกูลาร์เอ็กซ์เพรสชัน
year_	[0]*{nonzerodigit}{digit}*
date29	([0]?{nonzerodigit} [1-2]{digit})
month29	{date29}"/"[0]"2"/"{year_}
month31	({date29}"3"[0-1])"/"([0]?[13578]"10" "12")"/"{year_}
month30	({date29}"30")"/"([0]?[469]"11")"/"{year_}

date	({month29} {month30} {month31})
time_	([0-1]? {digit} "2"[0-3]) ":" [0-5]? {digit} ":" [0-5]? {digit}
time	"[" {time_} "]" {date} "
second	{integer} "sec"
minute	{integer} "min"
hour	{integer} "hr"
day	{integer} "day"
month	{integer} "mt"
year	{integer} "yr"

ตารางที่ 4-4 ตารางเรกกูลาร์เอ็กซ์เพรสชันสำหรับข้อมูลชนิดไทม์

4.2.3.2 การกระทำกับข้อมูลชนิดไทม์

เมื่อมีชนิดของข้อมูลที่ย่อมจะต้องมีการกระทำของข้อมูลชนิดนั้นๆ ด้วยซึ่งการกระทำของข้อมูลชนิดไทม์นั้นจะมีดังนี้

การกระทำกับข้อมูลชนิดไทม์		
Time Point	=	Time Point + Time Interval
Time Point	=	Time Point - Time Interval
Time Interval	=	Time Interval + Time Interval
Time Interval	=	Time Interval - Time Interval
Time Interval	=	Time Point – Time Point
Time Interval	=	Time Interval * Number
Number	=	Time Interval / Time Interval
Time Interval	=	Time Interval / Number
Time Interval	=	Time Interval ** Number
Number	=	Time Interval % Time Interval
Time Interval	=	Time Interval % Number

ตารางที่ 4-5 ตารางการกระทำลักษณะต่างๆ สำหรับข้อมูลชนิดไทม์

4.2.3.3 คำสั่งในการเปิดเว็บไซต์

คำสั่งในการเปิดเว็บไซต์เราจะต้องทำการออกแบบ บีเอ็นเอฟ ของคำสั่งนี้ซึ่งให้ชื่อของคำสั่งนี้ว่า openURL และมีรูปแบบการทำงานสองลักษณะดังต่อไปนี้

- เปิดเว็บไซต์ทันที openURL(<url>)
- เปิดเว็บไซต์ตามเวลาที่กำหนด openURL(<url>, <time>)

จากลักษณะของคำสั่งข้างต้นที่ได้กล่าวมาแล้ว จึงสามารถออกแบบ บีเอ็นเอฟ ตามลักษณะของคำสั่งการเปิดเว็บไซต์ได้เป็นดังนี้

ชื่อ	บีเอ็นเอฟ
url_stmt	URL '(' STRING ')' URL '(' STRING ',' TIME ')'

ตารางที่ 4-6 ตารางบีเอ็นเอฟของคำสั่งการเปิดเว็บไซต์

4.2.3.3 คำสั่งในการส่งเมลล์

คำสั่งในการส่งเมลล์เราจะต้องทำการออกแบบ บีเอ็นเอฟ ของคำสั่งนี้ซึ่งให้ชื่อของคำสั่งนี้ว่า sendMail และมีรูปแบบการทำงานสองลักษณะดังต่อไปนี้

- ส่งเมลล์ทันที sendMail(<String>, <String>, <String>, <String>)
- ส่งเมลล์ตามเวลาที่กำหนด openURL(<String>, <String>, <String>, <String>, <time>)

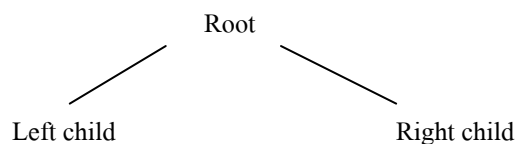
จากลักษณะของคำสั่งข้างต้นที่ได้กล่าวมาแล้ว จึงสามารถออกแบบ บีเอ็นเอฟ ตามลักษณะของคำสั่งการเปิดเว็บไซต์ได้เป็นดังนี้

ชื่อ	บีเอ็นเอฟ
mail_stmt	MAIL '(' STRING ',' STRING ',' STRING ',' STRING ')' MAIL '(' STRING ',' STRING ',' STRING ',' STRING ',' TIME ')'

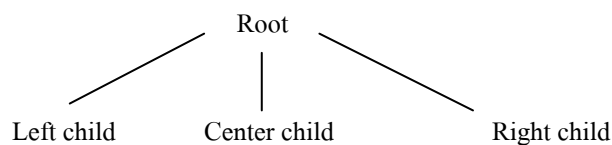
ตารางที่ 4-7 ตารางบีเอ็นเอฟของคำสั่งการส่งเมลล์

4.2.4 การออกแบบและสร้างพาร์ซทรี

การสร้างพาร์ซทรีนั้น จะเป็นการนำคำสั่งที่ผ่านการตรวจสอบประโยกว่าถูกต้องแล้วเท่านั้นถึงจะนำมาสร้างพาร์ซทรี โดยในการออกแบบนี้จะมองทรีออกเป็นสองลักษณะคือ ทรีที่มี 2 กิ่ง และ ทรีที่มี 3 กิ่ง



รูปที่ 4-2 แสดงพาร์ซทรีแบบสองกิ่ง

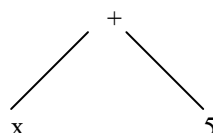


รูปที่ 4-3 แสดงพาร์ซทรีแบบสามกิ่ง

1. พาร์ซทรีแบบสองกิ่ง

สำหรับการออกแบบพาร์ซทรีแบบสองกิ่งนั้นก็จะสร้างไว้สำหรับใช้กับคำสั่งโดยทั่วไปที่กระทำตามตัวกระทำโดยตรง และไม่มีเงื่อนไขในการกระทำ หากเจอคำสั่งลักษณะเช่นนี้ เราก็นำเอาตัวกระทำเป็นรากของทรีส่วนที่เหลือก็จะเป็นลูกทางซ้ายและลูกทางขวาตามลำดับ เช่น

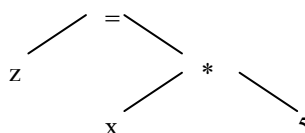
$x + 5$ จะได้พาร์ซทรีดังนี้



รูปที่ 4-4 แสดงตัวอย่างการสร้างทรีแบบสองกิ่งจากคำสั่งไพธอน $x + 5$

จากตัวอย่างจะเห็นได้ว่าการกระทำของคำสั่งคือเครื่องหมายบวก ซึ่งเราก็จะนำมาเป็นรากของทรี และทางด้านซ้ายของการกระทำก็คือ x เราก็จะนำมาเป็นลูกทางด้านซ้ายของทรี และในทำนองเดียวกัน 5 อยู่ทางด้านขวาของการกระทำเราก็จะนำมาเป็นลูกทางขวาของทรี

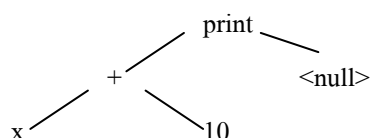
$z = x * 5$ จะได้พาร์ซทรีดังนี้



รูปที่ 4-5 แสดงตัวอย่างการสร้างทรีแบบสองกิ่งจากคำสั่งไพธอน $z = x * 5$

จากตัวอย่างจะเห็นได้ว่าการกระทำของคำสั่งคือการกำหนดค่า ซึ่งเราก็จะนำมาเป็นรากของทรี และทางด้านซ้ายของการกระทำก็คือ z เราก็จะนำมาเป็นลูกทางด้านซ้ายของทรี และในทำนองเดียวกัน $x * 5$ อยู่ทางด้านขวาของการกระทำเราก็จะนำมาเป็นลูกทางขวาของทรี แต่ว่าลูกทางด้านขวานั้นมีลักษณะเป็นทรีอีก ฉะนั้นเราก็ต้องมาทำการสร้างต่อโดยการกระทำของทรี $x * 5$ นั่นก็คือการคูณ เราก็จะนำมาเป็นรากของทรีและทางด้านซ้ายของการกระทำก็คือ x เราก็จะนำมาเป็นลูกทางด้านซ้ายของทรี และในทำนองเดียวกัน 5 อยู่ทางด้านขวาของการกระทำเราก็จะนำมาเป็นลูกทางขวาของทรี

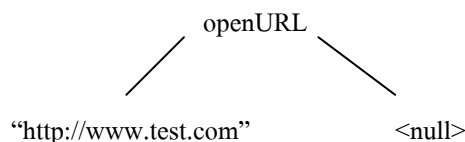
`print x + 10` จะได้พาร์ซทรีดังนี้



รูปที่ 4-6 แสดงตัวอย่างการสร้างทรีแบบสองกิ่งจากคำสั่งไพธอน `print x + 10`

จากตัวอย่างจะเห็นได้ว่าการกระทำของคำสั่งคือการแสดงผล (print) ซึ่งเราก็จะนำมาเป็นรากของทรี โดยที่คำสั่งการแสดงผลนั้นจะไม่มีลูกทางด้านขวา เพราะเป็นแค่การแสดงผลออกมาโดยที่ไม่ได้กระทำกับค่าใด จึงกำหนดให้ลูกทางด้านขวาเป็นค่า null ส่วนทางด้านซ้ายเป็นทรีของการบวก เราก็จะนำการบวกเป็นราก ส่วนลูกทางด้านซ้ายก็คือ z และ ลูกทางด้านขวาเป็น 5

`openURL("http://www.test.com")` จะได้พาร์ซทรีดังนี้



รูปที่ 4-7 แสดงตัวอย่างการสร้างทรีแบบสองกิ่งจากคำสั่งการเปิดเว็บไซต์

จากตัวอย่างคำสั่งการเปิดเว็บไซต์นั้น จะมีลักษณะเหมือนกับคำสั่งการแสดงผลทางหน้าจอที่ได้กล่าวไปแล้ว คือจะไม่มีลูกทางด้านขวาเนื่องจากไม่มีการกระทำกับค่าใด โดยจะมีเพียงลูกทางด้านซ้ายซึ่งก็คือยูอาร์แอล ที่ต้องการจะทำการเปิดนั่นเอง

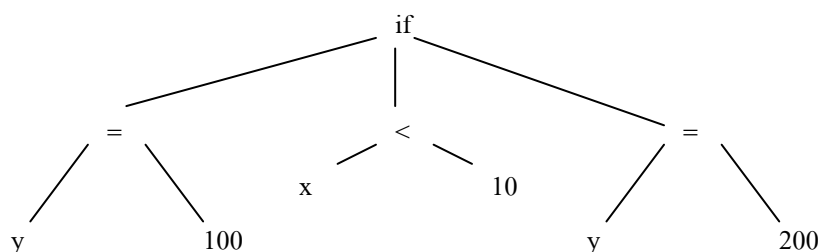
2. พาร์ซทรีแบบสามกิ่ง

สำหรับการออกแบบพาร์ซทรีแบบสามกิ่งนั้นก็จะสร้างไว้สำหรับใช้กับคำสั่งที่มีเงื่อนไขในการกระทำหรือไม่กระทำซึ่งทำให้มีผลของการกระทำมากกว่า 1 การกระทำหากเจอคำสั่งลักษณะเช่นนี้ เราก็จะนำเอาชื่อคำสั่งซึ่งจะบ่งบอกถึงการกระทำในภายหลัง มาเป็นรากของทรี ส่วนลูกทางด้านซ้ายนั้นจะให้เป็นที่ของคำสั่งที่จะถูกกระทำเมื่อเงื่อนไขเป็นจริง ส่วนลูกทางด้านขวาจะเป็นที่ของคำสั่งที่จะถูกกระทำเมื่อเงื่อนไขเป็นเท็จ และสุดท้ายก็นำเงื่อนไขของคำสั่งมาเป็นลูกตรงกลางของทรี เช่น

if $x < 10$: $y = 100$

else : $y = 200$

จะได้พาร์ซทรีดังนี้



รูปที่ 4-8 แสดงตัวอย่างการสร้างทรีแบบสามกิ่งจากคำสั่งไฟรอน $\text{if } x < 10 : y = 100 \text{ else : } y = 200$

จากตัวอย่างจะเห็นได้ว่าชื่อของคำสั่งนั้นก็คือ “if” ซึ่งเราก็จะนำมาเป็นรากของทรี และทางด้านซ้ายของทรีนั้น ก็จะเป็นที่ของคำสั่งที่จะกระทำเมื่อเงื่อนไขเป็นจริง ซึ่งนั่นก็คือ $y = 100$ ส่วนลูกทางด้านขวานั้นก็จะเป็นที่ของคำสั่งที่จะกระทำเมื่อเงื่อนไขเป็นเท็จ ซึ่งนั่นก็คือ $y = 200$ ส่วนสุดท้ายคือลูกตรงกลางนั้นก็จะเป็นเงื่อนไขของคำสั่งซึ่งก็คือ $x < 10$

โดยเราจะสังเกตได้ว่าลูกทั้งสามกิ่งนั้นจะเป็นที่ของแต่ละคำสั่งอีกทีซึ่งจะมีสองหรือสามกิ่งต่อไปอีกหรือไม่ก็ขึ้นอยู่กับคำสั่งหรือการกระทำนั้นๆ จะเป็นอะไรนั่นเอง

if $x < 10$:

$z = 100$

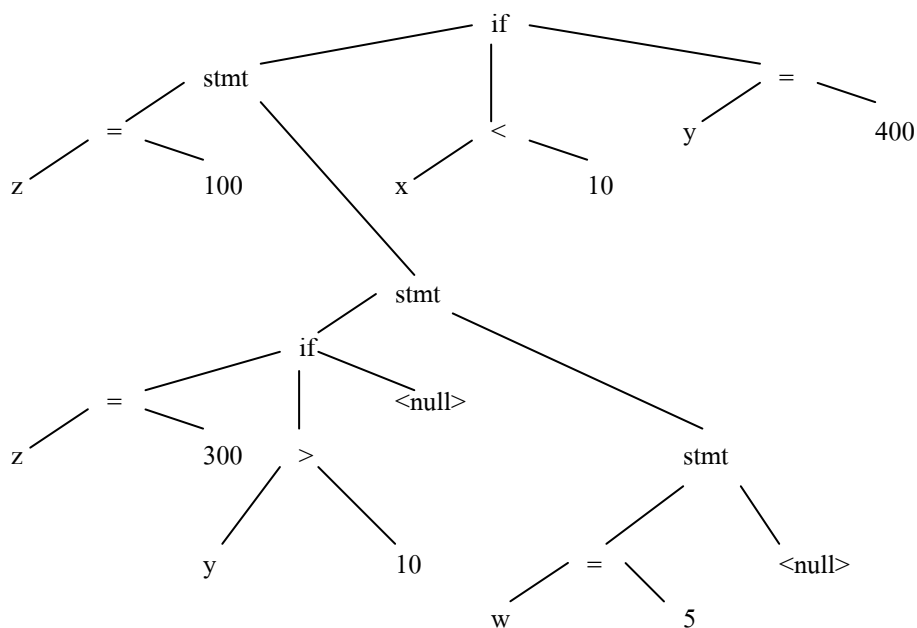
if $y > 10$:

$z = 300$

$w = 5$

else : $y = 400$

จะได้พาร์ซทรีดังนี้



รูปที่ 4-9 แสดงตัวอย่างการสร้างพาร์ซทรีแบบสามกิ่งจากคำสั่งไพธอนที่ซ้อนหลายชั้น

จากตัวอย่างนี้จะแสดงให้เห็นถึงการสร้างพาร์ซทรีแบบสามกิ่งซ้อนกันหลายๆ ชั้นและภายใต้คำสั่ง “if” จะมีหลายคำสั่งอยู่ภายในคำสั่ง “if” ซึ่งจะสังเกตเห็นได้ว่าหากมีคำสั่งเพียงคำสั่งเดียวอยู่ภายใต้คำสั่ง “if” กิ่งทางด้านซ้ายของคำสั่งจะเป็นการกระทำของคำสั่งที่อยู่ภายใต้ “if” ทันทีแต่หากมีคำสั่งมากกว่า 1 คำสั่งกิ่งทางด้านซ้ายนั้นจะเป็นพาร์ซทรีที่ชื่อว่า “stmt” ซึ่งจะบรรจุคำสั่งไว้ทางด้านซ้ายของพาร์ซทรีและทางด้านขวา ก็จะเป็นคำสั่งถัดไปเรื่อยๆ จะกระทั่งลูกทางด้านขวาของ “stmt” เป็น null ซึ่งแสดงว่าไม่มีคำสั่งต่อจากนี้แล้ว

และจากตัวอย่างให้สังเกตคำสั่ง “if” ที่อยู่ภายใต้คำสั่ง “if” อีกทีนั้น จะไม่มีส่วนของ “else” ซึ่งทำให้ลูกทางด้านขวาของตัวมันเป็น null

4.3 การออกแบบและสร้างส่วนของการกระทำตามคำสั่ง

ส่วนของการกระทำตามคำสั่งนั้นเป็นส่วนสุดท้ายของกระบวนการการอินเทอร์พรีต ซึ่งถ้าดูเผินๆ แล้วอาจเห็นว่าส่วนของการกระทำตามคำสั่งนั้นก็เพียงแค่กระทำไปตามลักษณะของคำสั่งที่ได้รับมาจากพาร์ซทรี แต่จริงๆ แล้วหน้าที่ของส่วนนี้จะมียู่ด้วยกันสามส่วนก็คือ

- ตรวจสอบความผิดพลาดของชนิดตัวแปรที่นำมากระทำตามคำสั่งต่างๆ
- กระทำตามคำสั่งที่รับเข้ามา
- จัดการกับตัวแปรต่างๆ ที่ต้องนำไปกระทำ หรือได้จากการกระทำนั้นๆ

4.3.1 การตรวจสอบความผิดพลาดของชนิดตัวแปรที่นำมากระทำตามคำสั่ง

เนื่องจากการ สร้างเรกกูลาร์เอ็กซ์เพรสชัน และการสร้าง บีเอ็นเอฟนั้นจะเป็นการกำหนดความสัมพันธ์ระหว่างกลุ่มคำและประโยคของภาษา โดยจัดเป็นกลุ่มๆ เป็นพวกรๆ เท่านั้น ไม่สามารถที่จะจัดการได้ถึงขั้นเป็นคำๆ หรือเป็นตัวๆ ได้ หรือหากทำได้ก็จะทำให้มีความซับซ้อนในการทำงานมากจนเกินไปได้

โดยการกระทำดังกล่าวนี้สามารถยกหน้าที่ให้มาทำในส่วนของการกระทำตามคำสั่งได้ เพราะว่าในส่วนนี้นั้น จะเข้าไปกระทำยังข้อมูลที่ลึกที่สุดของแต่ละส่วนของคำสั่ง ซึ่งจะทำให้สามารถทราบได้ว่าการกระทำนั้นสามารถกระทำได้หรือไม่ เพื่อให้เข้าใจมากขึ้นจะยกตัวอย่างให้เห็นลักษณะดังกล่าวดังต่อไปนี้

สมมติเรากำหนดให้

literal → INTEGER | STRING | LONGINTEGER

และ

expression → literal '+' literal

ซึ่งนั่นหมายความว่า การกระทำการบวกนั้นสามารถทำได้โดยการนำเอา literal บวกกับ literal แต่ว่าใน literal เองนั้นมีตัวแปรอยู่ด้วยกัน 3 ชนิดคือ integer, string และ long integer หากในการใช้คำสั่งจาก บีเอ็นเอฟ นี้เป็น $10 + 20$ ซึ่งก็เป็น integer บวกกับ integer ซึ่งก็จะมีปัญหาอะไร แต่หากลองเขียนใหม่เป็น "test" + 10 อย่างนี้ การตรวจสอบประโยคนั้นก็ไม่สามารถตรวจพบได้ว่ามีความผิดพลาดเกิดขึ้นเนื่องจาก string และ integer ต่างก็ถูกแทนด้วย literal และการบวกก็สามารถทำได้โดยการนำเอา literal บวกกับ literal

จึงทำให้เราเห็นได้ว่า จำเป็นอย่างยิ่งที่ส่วนของการกระทำตามคำสั่งนั้นจะต้องทำการตรวจสอบชนิดของข้อมูลก่อนว่าสามารถจะกระทำตามคำสั่งนั้นๆ ได้หรือไม่

4.3.2 การกระทำตามคำสั่งที่รับเข้ามา

หลังจากที่มีการตรวจสอบได้แล้วว่าชนิดของข้อมูลที่จะถูกกระทำนั้นไม่ขัดแย้งกับการกระทำเราก็สามารถที่จะกระทำตามคำสั่งนั้นได้ทันที โดยการเชื่อว่าเป็นการกระทำใด ก็กระทำไปตามนั้นได้ทันที

4.3.3 จัดการกับตัวแปรต่างๆ ที่ต้องนำไปกระทำ หรือได้จากการกระทำ

เนื่องจากการกระทำต่างๆ ส่วนใหญ่แล้วมักจะไม่ใช่เป็นการกระทำระหว่างค่าคงที่กับค่าคงที่ แต่มักจะเป็นการกระทำระหว่าง ตัวแปรกับค่าคงที่หรือไม่ก็ตัวแปรกับตัวแปรมากกว่า อีกทั้งผลที่ได้จากการกระทำต่างๆ ก็มักจะนำไปเก็บไว้ในตัวแปรด้วย

ดังนั้นส่วนของการกระทำตามคำสั่งนี้ จะต้องมีการจัดการกับตัวแปรเหล่านี้ โดยจะแบ่งการจัดการกับตัวแปรต่างๆ ได้เป็นสองกรณีคือ

- นำค่าที่ได้จากการกระทำไปเก็บไว้ในตัวแปร
- นำค่าที่เก็บไว้ออกมาใช้เพื่อกระทำคำสั่งต่างๆ

โดยการจัดการทั้งสองกรณีนี้ ก็จะต้องมีสิ่งที่ใช้ในการเก็บและนำออก ซึ่งเราเรียกว่าตารางสัญลักษณ์ (Symbol Table) ซึ่งก็จะหมายถึงตารางที่ใช้เก็บตัวแปรและค่าของตัวแปรต่างๆ ที่เกิดขึ้นในขณะทำงานโปรแกรมหนึ่งๆ หรือ สคริปต์หนึ่งๆ นั่นเอง โดยเราจะสร้างขึ้นโดยใช้ ตารางแฮช (Hash Table) ซึ่งมีคุณสมบัติที่เหมาะสมกับการเก็บและนำค่าที่เก็บไว้กลับมาใช้รวมถึงการเปลี่ยนแปลงค่าที่เลขมืออยู่เดิมเป็นค่าใหม่